ACCESSWINDOW · NETPROXY

Project Topics

Three sub-topics that plug into each other — each with what you built, its interface, a demo and viva questions.

Topic 1

Secure Proxy Gateway

sockets · TLS · identity · limits

Member: _____

Topic 2

Access Policy & Filtering Engine

rules · filters · live reload

Member: _____

Topic 3

Access Management Dashboard

Flask · roles · requests · demo lab

Member: _____

Question numbers match **STUDY_GUIDE.pdf**

Contents

Overview — three sub-topics at a glance

Topic 1 — Secure Proxy Gateway

Topic 2 — Access Policy & Filtering Engine

Topic 3 — Access Management Dashboard

Shared — everyone answers these

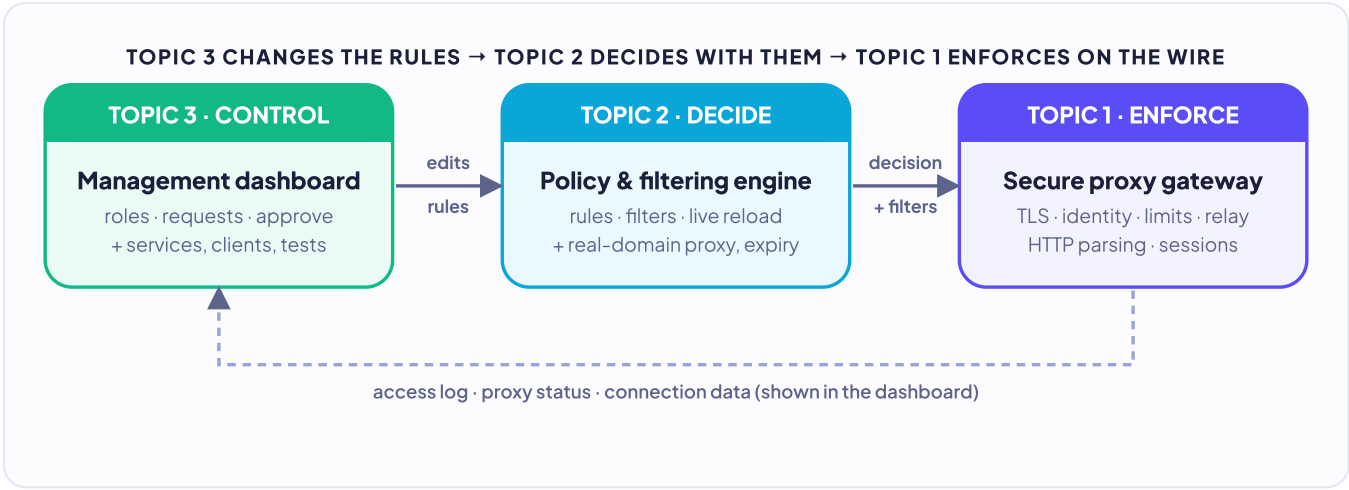
Presenting together

The project is divided into **three sub-topics that plug into each other**. Each member claims one, owns it completely, and can say *"I built this"* — with a standalone demo and a clear interface to the other two. Claim only what you can genuinely explain; an examiner will ask follow-ups. Question numbers (Q1…Q54) match `STUDY_GUIDE.pdf`, so you can read the full background there.

Member 1: `_____` **Member 2:** `_____` **Member 3:** `_____`

Overview — three sub-topics at a glance

	Topic 1 · Secure Proxy Gateway	Topic 2 · Access Policy & Filtering Engine	Topic 3 · Access Management Dashboard
One line	The secure gateway every connection passes through: sockets, TLS, identity, limits, relay	The brain that decides what is allowed: rules, content filters, live updates, real-domain blocking	The control panel: web app, roles, request → approve → expire, plus the demo lab
Main files	`netproxy/server.py` · `httpmsg.py` · `registry.py` · `tlsutil.py` · `config.py` · `proxy.py` · `gen_certs.py` · `blockpage.py`	`netproxy/policy.py` · `filters.py` · `control.py` · `accesswindow.py` · `forward_proxy.py` · `adminctl.py`	`app.py` (incl. Browse) · `templates/` · `static/` · `services.py` · `client.py` · `netproxy/clientlib.py` · `testbed.py`
Code size	≈ 1,830 lines	≈ 1,620 lines	≈ 1,820 lines Python + UI templates/CSS
Tests you can run	`test_integration.py` (19)	`test_policy.py` (24) · `test_filters.py` (14) · `test_forward_proxy_policy.py` (4)	`test_accesswindow_roles.py` (4) · `test_site_temp_access.py` (4) · `demo.py`
Key ideas	TCP, threads, select, TLS, mutual TLS, CONNECT, limits	ACL first-match, stages, normalisation, filters, live reload, expiry	Flask, sessions, roles, CSRF, request→approve→revoke, UI
Viva questions	Q5–Q11, Q25–Q28, Q43, Q49–Q51	Q12–Q24, Q36–Q38, Q44, Q46, Q52–Q53	Q29–Q35, Q39–Q42, Q45, Q47–Q48, Q54
Pages in the PDF	6 – 10	11 – 16	17 – 21



How the three topics integrate

Each topic exposes one small, well-defined **contract**. That is why they can be built and demonstrated separately and still work as one system.

Contract	Defined by	Used by	Shape
Policy document	Topic 2 (`policy.py`)	Topic 1 loads it · Topic 3 edits it	JSON: `services` , `groups` , `default_action` , `rules[]` , `content_filters[]` , `limits`
Decision API	Topic 2	Topic 1	`Policy.evaluate(Context, stage) → Decision(allowed, rule_id, reason)`
Identity	Topic 1 (`tlsutil.py`)	Topic 2	certificate Common Name → `Context.user` (`anonymous` if no certificate)
Admin protocol	Topic 2 (`control.py`)	Topic 3 (`AdminClient`)	TLS line protocol: `AUTH <token>` , then `RULE INSERT <n> <json>` , `RULE DEL <id>` , `STATUS` , `LOG <n>` ; every reply starts `OK` / `ERR` and ends with `.`
Real-domain rule	Topic 2 (`match_rule`)	Topic 3 writes it	`{id, action, users[], domains[], enabled, temporary?, expires_at?}` in `real_website_policy.json`
Live revalidation	Topic 2 (policy swap notifies listeners)	Topic 1 acts on it	`Registry.revalidate()` terminates sessions the new policy denies
Block page	Topic 2 hands over the matched rule and user (`forward_proxy.py`)	Topic 1 serves it over TLS (`blockpage.py`) · Topic 3 receives the click	The page shows reason, author and date from the rule; its button opens the dashboard at `/requests?domain=<site>`
Access logs	Topic 1 and Topic 2's forward proxy	Topic 3 displays them	JSON lines with time, user, destination, decision, rule id, reason

The integrated story (what you show when the three topics run together)

1. **Topic 3** — the administrator blocks `youtube.com` in the dashboard; it writes the rule.
2. **Topic 2** — Bob's browser asks the forward proxy for `youtube.com`; `match_rule` finds the deny rule and the attempt is logged. With the block page on, **Topic 1's** TLS code answers the blocked request with a page showing the reason, who restricted it and when, and a **Request temporary access** button that opens Topic 3's request form with the site selected.
3. **Topic 3** — the blocked attempt appears in the dashboard's alerts; Bob requests 5 minutes; the administrator approves → an allow rule with `expires_at` is inserted at the top.
4. **Topic 2** — the next request matches the new rule first and passes; at the deadline `rule_expired()` makes the deny rule win again.
5. **Topic 1** — for the lab services: Bob's chat client connects over TLS, his certificate gives the identity, Topic 2 decides, and Topic 1 relays the bytes while Topic 2's filters scan each line.
6. **All three** — the administrator runs `BLOCK user bob` from the dashboard or `adminctl`: Topic 2 swaps in the new policy and Topic 1 cuts Bob's live session at once.

The real-domain proxy is small and self-contained, so it lives with Topic 2 and does its own relaying; the lab services go through Topic 1's gateway.

Everyone should be able to say, in one breath: *"Topic 3 changes the rules, Topic 2 decides with them, Topic 1 enforces the decision on the wire."*

Topic 1 — Secure Proxy Gateway

Your 30-second pitch

"I built the network layer. It accepts client connections, secures them with TLS, identifies each user from their client certificate, applies connection and rate limits, parses HTTP strictly, connects to the back-end service, and relays the bytes. At each decision point it asks the policy engine — which is Topic 2 — whether to continue."

What you can claim — "I designed and built…"

- the **TLS gateway**: listener, accept loop and a hard cap on simultaneous connections;
- **mutual-TLS identity** — the certificates (`gen_certs.py`) and reading the user from the client certificate;
- **per-user connection and rate limits**, plus handshake / idle / backend timeouts;
- a **strict HTTP parser** that refuses request-smuggling tricks;
- the **tunnel and relay engine** in three modes (raw bytes, line-inspected, HTTP-aware) with half-close and live termination;
- the **live connection table and access log**;
- the **block page's TLS side** (`blockpage.py`): a dedicated local CA, per-host certificates made with `openssl` , and serving the page over TLS for **blocked hosts only**.

Interface — what you expose and consume

You expose	You consume
TLS port `:8443` accepting `CONNECT <service>` or an absolute `http://` request	`Policy.evaluate(Context)` → *allow / deny + rule id* (Topic 2)
Identity of each connection (certificate Common Name, or `anonymous`)	`filters_for(user, service, direction)` (Topic 2)
Access log (JSON lines) and the live connection table	`revalidate()` calls after a policy change (Topic 2)

Standalone demo: run `services.py` + `proxy.py` with the shipped policy and use `client.py` — Topic 1 works on its own with a fixed policy file.

Files you own

File	What it does
`netproxy/server.py`	`ProxyServer` (listeners + accept loop, semaphore cap) · `ClientSession` (one thread per client: handshake → admit → read request → policy check → backend → relay) · `Channel` (non-blocking socket wrapper).

File	What it does
`netproxy/httpmsg.py`	Strict HTTP/1.1 parsing: request/response heads, body framing, absolute-URI split, header forwarding, error and block pages.
`netproxy/registry.py`	Live connection table, `admit()` (per-user cap + sliding-window rate limit), counters, access log, `revalidate()`.
`netproxy/tlsutil.py`	TLS server/client contexts; `peer_common_name()` turns a client certificate into the user's **identity**.
`netproxy/config.py`, `config/proxy.json`	Listener ports, timeouts, limits, certificate paths.
`proxy.py`	Start-up wiring: load config and policy, start server, admin console, expiry worker.
`gen_certs.py`	Creates the CA, proxy certificate and client certificates.
`blockpage.py`	Block page for blocked HTTPS sites: dedicated CA (`certs/blockpage/`), short-lived per-host certificates via `openssl`, the "why / by whom" page, `Clear-Site-Data`. Hooked into `forward_proxy.py` with `--block-page`.

Concepts you must explain

- **Sockets and threads:** thread per client; `BoundedSemaphore(max_connections)` rejects overload immediately.
- **Non-blocking I/O with `select`:** `Channel.try_recv()` asks the socket first because an `SSLSocket` may hold already-decrypted bytes `select()` can't see.
- **TLS handshake and certificates:** CA signs the proxy and client certificates; minimum TLS 1.2; `CERT_OPTIONAL` vs `CERT_REQUIRED`.
- **Mutual TLS identity:** certificate Common Name → `c.user`; no certificate → `anonymous`. Identity cannot be faked by a header.
- **TLS interception for the block page:** a browser ignores a refused `CONNECT`'s body, so the proxy answers `200` and completes TLS itself with a certificate signed by its own CA — which the browser must trust once. Only blocked hosts are intercepted; allowed traffic stays end-to-end encrypted. Risk: whoever holds the CA key can mint trusted certificates, so the CA is dedicated, locked down and removable.
- **CONNECT tunnel:** `CONNECT service HTTP/1.1` → `200 Connection Established` → relay. Half-close (`shutdown_write`) is passed through.
- **Limits and timeouts:** concurrent cap, sliding-window rate limit (`deque`), handshake / idle / backend timeouts, header and body size limits.
- **Request smuggling defence:** the parser rejects both `Content-Length` and `Transfer-Encoding`, conflicting lengths and whitespace before `:`.
- **Terminating a live session:** `kill()` sets a `threading.Event`; the relay loop checks it every ≤ 0.1 s and raises `Terminated`.

Walk through your code (say it in this order)

1. `ProxyServer._accept_loop` accepts → acquires a slot → starts a thread running `ClientSession.run()`.
2. `_handshake()` completes TLS and reads the certificate's Common Name as the user.
3. `registry.admit()` enforces the per-user cap and rate limit (`429` if exceeded).
4. `_serve()` reads the request head; `httpmsg.parse_request_head` validates it.
5. `_connect()` resolves the service and **asks Topic 2** `policy.evaluate(..., stage="connect")`; a denial becomes `403` with the rule id.
6. `_open_backend()` connects to the service; the proxy replies `200 Connection Established`.
7. `_relay()` (raw or line mode) or `_http_session()` (per-request) moves the data, calling Topic 2's filters on the way.
8. `_cleanup()` removes the connection from the registry and writes the log.

Live demo

```
python3 services.py                       # terminal 1
python3 proxy.py                          # terminal 2
python3 client.py --user bob web /news    # bob's certificate → allowed, X-Proxy-User: bob
python3 client.py web /admin/panel        # anonymous → refused by a rule (shows rule id in the reply)
python3 adminctl.py CONNS                 # live connections: user, service, bytes
python3 -m unittest tests.test_integration -v
```

Hand-off lines (when a question belongs to another part)

- *"Which rule matched / how rules are ordered?"* → "That's the policy engine — Topic 2. From my side, I call `evaluate()` and enforce the answer."
- *"How does the admin approve access?"* → "That's the dashboard — Topic 3. It reaches me only through the admin console and the policy."

Your viva questions

Q5. What is `CONNECT` and why is it used for HTTPS?

It asks the proxy to open a raw TCP tunnel to `host:port`. The proxy can't parse HTTPS, so after `200 Connection Established` it just relays encrypted bytes.

Q6. Can the proxy see which page I visit on an HTTPS site?

No — only the hostname from `CONNECT`. The path and content are inside TLS.

Q7. What is mutual TLS and why use it?

Both sides present certificates. The client certificate's Common Name becomes its identity, which can't be faked by editing a header. Without a certificate the user is `anonymous`.

Q8. What is a CA and what does `gen_certs.py` create?

A Certificate Authority signs certificates. The script makes a CA, the proxy's server certificate and client certificates; clients verify the proxy (name `netproxy`) against the CA.

Q9. Why TCP_NODELAY?

It disables Nagle's algorithm so small interactive messages (chat) aren't delayed.

Q10. What is a half-close and why do you handle it?

One side finishes sending but can still receive. `shutdown_write()` passes this through so protocols like echo or upload complete correctly (`test_half_close_through_tunnel`).

Q11. Why does `Channel.try_recv` call `recv()` before `select()`?

An `SSLSocket` may already hold decrypted data in its buffer; `select()` only sees the raw socket and would wait for data that has already arrived.

Q25. How does the server handle many clients?

Thread per connection, capped by a `BoundedSemaphore`; non-blocking sockets with `select`.

Q26. How is a live session terminated?

`kill()` sets a `threading.Event`; the relay loop checks it every ≤ 0.1 s and raises `Terminated`.

Q27. What are idle and handshake timeouts for?

They stop slow or stuck clients (slowloris) from holding resources.

Q28. How are rate limits implemented?

Per identity: a current-connection counter plus a `deque` of recent connection times (sliding window). Exceeding either returns `429`.

Q43. What if the user turns off the proxy setting?

Filtering is bypassed — a proxy only filters traffic sent to it. Real enforcement needs network-level controls.

Q49. Why does a blocked HTTPS site show "site can't be reached" instead of a message?

The browser asks for a tunnel (`CONNECT`) before it starts TLS. When the proxy refuses, browsers discard the refusal's body and show their own error page.

Q50. How does your block page work for HTTPS?

With `--block-page` the proxy answers the `CONNECT` with `200` and completes TLS itself, using a certificate for that host signed by a dedicated local CA (made with `openssl`). If the browser trusts the CA, it shows our page — the reason, who restricted it, and a request button. Only blocked hosts are intercepted.

Q51. Isn't that dangerous? What are the risks and how do you limit them?

While trusted, the CA key could sign certificates for any site on that machine. Mitigations: a dedicated CA, key `chmod 600` and git-ignored, short-lived (30-day) leaf certificates, strict host validation (no IPs), only blocked hosts intercepted, and the trust is removed after the demo.

Topic 2 — Access Policy & Filtering Engine

Your 30-second pitch

"I built the decision-making. A policy is an ordered list of rules — first match wins — plus content filters and limits. I evaluate it at two stages, connect and request, normalise paths so rules can't be bypassed, inspect content as a stream, and reload the policy live. I also built the real-domain forward proxy and the expiry logic that makes temporary access end by itself."

What you can claim — "I designed and built…"

- the **rule engine**: first-match ordering, users / groups / exceptions, source networks, services, domains, paths, methods and **time schedules** (including windows past midnight);
- **two-stage evaluation** (connect vs request) and **path normalisation** against bypass tricks;
- the **content filters** — keyword, regex and MIME, with block / redact / log — and the **stream inspector** that catches keywords split across packets;
- an **immutable, versioned policy** with atomic hot-reload and **live revalidation** of open connections;
- the **admin console** protocol;
- the **real-domain forward proxy** — authentication, hostname and subdomain matching, tunnelling — and **self-expiring rules** that make temporary access end by itself;
- **related domains** (`youtube.com` also covers `googlevideo.com` , `ytimg.com` …) and a **live re-check of open tunnels** every second, so a revoke cuts a connection that is already open;
- the **block-page hook**: on a deny, hand the matched rule and user to the block page.

Interface — what you expose and consume

You expose	You consume
`Policy.evaluate(Context) → Decision(allowed, rule_id, reason)` and `filters_for()`	`Context(user, src, service, method, path, domain)` built by Topic 1
Admin console (TLS line protocol): `AUTH` , `RULE INSERT / DEL` , `STATUS` , `LOG` , `BLOCK` …	Rule edits sent by Topic 3 (admin commands, or writes to `real_website_policy.json`)
Real-domain rule format `{id, action, users, domains, enabled, temporary, expires_at}`	The policy JSON files

Standalone demo: the unit tests, `adminctl.py` , and `curl` through `forward_proxy.py` — no dashboard needed.

Files you own

File	What it does
`netproxy/policy.py`	Validates the JSON policy into immutable objects (`Rule` , `Schedule` , `UserMatcher` , `ContentFilter` , `Limits` , `Policy`). `Policy.evaluate()` is first-match. `PolicyStore` swaps versions atomically, watches the file, notifies listeners.
`netproxy/filters.py`	`inspect_bytes()` (keyword/regex), `check_mime()` , and `StreamInspector` (reassembles a TCP stream into lines before scanning).
`netproxy/control.py`	The admin console (TLS listener, token): `BLOCK` , `RULE` , `FILTER` , `KILL` , `STATUS` , `LOG` … every edit → validate → swap → revalidate.
`netproxy/accesswindow.py`	Expiry worker thread that removes expired temporary rules for the simulated services.
`forward_proxy.py`	Real-domain HTTP/HTTPS proxy: authentication, `match_rule()` , `rule_expired()` , tunnel with deadline, JSON logging.
`adminctl.py`	Command-line client for the admin console.
`config/policy.json` , `config/real_website_policy.json`	The two rule sets.

Concepts you must explain

- **ACL, first match wins:** order *is* the logic. A temporary allow goes **above** the deny.
- **Default action:** Layer A `deny` (whitelist); Layer B `allow` (blacklist).
- **What a rule can match:** user / `@group` / `anonymous` (with exceptions), source CIDR, service, domain, path glob, HTTP method, time schedule (incl. windows crossing midnight).
- **Connect vs request stage:** path rules can only be decided per request; at connect time a deny is skipped and an allow admits *provisionally*.
- **Path normalisation:** `/x/../admin` , `//admin` , `/%61dmin` can't evade `/admin/*` ; the proxy forwards the normalised path.
- **Content filters:** keyword / regex / mime × block / redact / log; block wins, then redact, then log.
- **Why a stream inspector:** keywords can straddle packets; scan complete lines and hold back a possible keyword prefix.
- **Dynamic policy:** immutable `Policy` , atomic swap, file watcher, invalid edits rejected, `revalidate()` kills newly denied live connections.
- **Real-domain matching:** `host == domain or host.endswith("." + domain)` — blocks subdomains, not `notyoutube.com` . A small family table makes `youtube.com` also cover `googlevideo.com` , `ytimg.com` , `youtu.be` , so blocked YouTube video can't keep playing.
- **Expiry and revocation in the data path:** `rule_expired()` is checked on every request, and open tunnels re-check their rule every second — so a deadline or a revoke closes them.

Walk through your code (say it in this order)

1. `Policy.__init__` validates the document and builds `Rule` objects (errors name the exact field).
2. `Policy.evaluate(ctx, stage)` loops over rules → `matches_base()` (user, network, service, domain, schedule) → `matches_request()` (method, path) → first match returns a `Decision(allowed, rule_id, reason)`.
3. `normalize_path()` canonicalises the URL before matching.
4. `StreamInspector.feed()` → `_process()` → `inspect_bytes()` returns a `FilterOutcome` (data, blocked filter, hits).
5. `PolicyStore._swap()` → listeners → `Registry.revalidate()` terminates now-denied sessions.
6. **Real-domain proxy:** `Handler.authenticate()` → parse target → `match_rule()` → `403` or tunnel (`tunnel(upstream, deadline)`).
7. **Expiry:** `rule_expired()` skips rules past `expires_at`; `accesswindow.py` removes service grants every second.

Live demo

```
python3 adminctl.py RULES                    # list the rules in order
python3 adminctl.py BLOCK user bob           # bob's open sessions are cut immediately
python3 adminctl.py UNBLOCK DYN-1
python3 forward_proxy.py                     # then, with youtube.com blocked in the dashboard:
curl -x http://127.0.0.1:8080 --proxy-user bob:bob-demo https://youtube.com/ -I     # 403
python3 -m unittest tests.test_policy tests.test_filters tests.test_forward_proxy_policy -v
```

Edit `config/policy.json` while the proxy runs and watch it reload within a second.

Hand-off lines

- *"How is the connection accepted / TLS done?"* → "That's the network core — Topic 1. I'm called after identity is known."
- *"Who creates the temporary rule?"* → "The dashboard — Topic 3 — inserts it; my code is what makes it match first and expire."

Your viva questions

Q12. What does "first match wins" mean and why does order matter?

Rules are tested top to bottom and the first whose conditions all match decides. So a temporary *allow* must sit above the *deny* to override it.

Q13. What is the default action and what's yours?

What happens when no rule matches. Layer A: `deny` (whitelist). Layer B: `allow` (blacklist).

Q14. What can a rule match on?

Users/groups (`@staff` , `*` , `anonymous` , with `except`), source network (CIDR), service, domain, path (glob), HTTP method, and a time schedule (days + hours, including windows crossing midnight).

Q15. Connect stage vs request stage?

Connect: identity, IP, service and time are known. Request: method and path are known too. Path-scoped rules can only be finally decided per request; at connect time a deny is skipped and an allow admits provisionally.

Q16. Why is `Policy` immutable and swapped atomically?

So a connection thread can never observe a half-updated rule list. Every change builds a new validated `Policy` (version + 1) and swaps the reference under a lock; invalid changes are rejected.

Q17. How do rule changes apply without restarting?

A watcher thread polls the file's modification time each second; admin commands use `store.mutate()` . After each swap `revalidate()` re-checks open connections and kills the ones now denied.

Q18. How does the schedule rule work?

`Schedule.active(now)` compares minutes-since-midnight and weekday; if `start > end` the window crosses midnight and belongs to the day it starts on.

Q19. What is path normalisation and why is it needed?

It prevents `/%61dmin` , `//admin` and `/x/../admin` from evading a rule for `/admin/*` while the back end still serves the page.

Q20. Difference between block, redact and log?

Block refuses (403 or terminates the stream), redact replaces the matched text and continues, log only records. If a block matches it wins outright.

Q21. Why can't you just scan each `recv()` chunk for keywords?

TCP has no message boundaries; a keyword can straddle two chunks. `StreamInspector` buffers into lines and holds back a tail that could begin a keyword.

Q22. What is the "fast path" in `_process` ?

One regex scan over the whole chunk; if nothing matches, forward it untouched. Only on a hit does it inspect line by line.

Q23. How do you stop executables being downloaded?

A `mime` filter (F04) checks the response `Content-Type` ; staff are exempt via `except_users` .

Q24. Why strip `Accept-Encoding` ?

A gzip-compressed body can't be scanned, so the proxy asks the server for identity encoding.

Q36. How does a site "re-block" itself after expiry?

`match_rule` skips rules where `rule_expired()` is true; the older deny underneath matches again. It is enforced in the data path, so no job is needed.

Q37. What happens to a tunnel that is open when the time runs out — or when access is revoked?

`tunnel()` wakes every second, stops at the grant's `deadline` , and re-runs `match_rule` for the tunnel's host and user, so a revoke or a new deny closes it within about a second.

Q38. Why insert the grant at index 0 instead of appending?

First-match-wins: appended, the older deny would match first and the grant would never apply.

Q44. Can the proxy block a single YouTube video?

Not over HTTPS — it only sees `youtube.com` . It would need TLS interception of *allowed* traffic too (the block page intercepts only blocked hosts).

Q46. What if the policy file is invalid?

It is rejected with an error; the previous version keeps running.

Q52. Why did YouTube show "Connect to the Internet" with a Downloads page when blocked?

YouTube's **service worker** serves a cached offline page when the network fails. The block page unregisters that worker and clears the origin's caches and storage (`Clear-Site-Data` plus a script), so the offline page and its downloads can't open. Cookies are kept.

Q53. Why did YouTube keep playing after you revoked access?

Two reasons, both fixed. Video streams from `googlevideo.com`, which wasn't in the rule — now `DOMAIN_FAMILIES` makes `youtube.com` cover its related domains. And revoking didn't close tunnels that were already open — now each tunnel re-checks its rule every second and closes within about a second of a revoke.

Topic 3 — Access Management Dashboard

Your 30-second pitch

"I built the control plane and the lab around it. The Flask dashboard has two roles: an administrator manages website rules and approves requests; a requester can ask for a service or a blocked site for a few minutes. Approval inserts a time-limited rule at the top of the policy. I also built the simulated services and clients used for demos and tests, the UI, and the security around the web app."

What you can claim — "I designed and built…"

- the **dashboard** with separate **administrator and requester roles** — login, hashed passwords, CSRF protection, hardened session cookie;
- the **request → approve → revoke workflow**, for services and for **websites (temporary unblock)**, with validation (only sites actually blocked for that user; no duplicates; no self-approval);
- a **page per menu item**, the account menu, the **live proxy status light**, countdown timers and the new visual design;
- the **demo lab**: simulated services (web, chat, echo, files, bulk), the command-line client, the in-process test bed and scripted demo;
- the **Browse page**: type a site and see the policy decide — a block card with a request form, or a safe scripts-off preview — with SSRF safeguards;
- the **request deep link**: the block page's button opens `/requests?domain=<site>` pre-filled, and login returns you there;
- the **tests** for roles, temporary access, Browse and the request link, the **animated packet diagram** and the **study guide**.

Interface — what you expose and consume

You expose	You consume
The web UI on `127.0.0.1:5000` for administrators and requesters, including Browse and the `/requests?domain=` link used by the block page	Admin console of Topic 2: `RULE INSERT`, `RULE DEL`, `STATUS`, `LOG` over TLS + token
Pending requests, grants and history (`accesswindow_requests.json`, `accesswindow_state.json`)	`real_website_policy.json` (read and write) and the proxy logs (to show alerts)
Simulated services and clients for demos and tests	Identity and sessions from Topic 1 when clients use the services

Standalone demo: the dashboard runs with the proxy offline (the status light turns red) and its tests use Flask's test client — no running proxy needed.

Files you own

File	What it does
`app.py`	Flask routes, login, roles, CSRF, request / approve / revoke logic, website rules, `dashboard_data()` + `render_page()`.
`templates/`	`base.html` (layout: sidebar, account menu, status light, proxy pill) and one page each: overview, requests, grants, activity, website access, login.
`static/`	`style.css`, `theme.css` (palette + typography), `app.js` (countdowns, account menu).
`services.py`	The five simulated back-end servers (web, chat, echo, files, bulk) built on the raw `socket` API.
`client.py`, `netproxy/clientlib.py`	Command-line client and the library behind it (also used by the dashboard's `AdminClient`).
`netproxy/testbed.py`, `demo.py`	Start the whole stack in-process on free ports for tests and the scripted demo.
`packet_flow.html`, `STUDY_GUIDE.pdf`	Animated diagram and the written guide.

Concepts you must explain

- **Roles enforced server-side:** `@role_required("admin")` returns `403`; hiding buttons is not security.
- **CSRF protection:** a per-session token in every POST form, checked with `hmac.compare_digest`.
- **Passwords:** salted PBKDF2-SHA256 hashes, never plaintext. (Why not scrypt: this Python build lacks it.)
- **Session cookie:** `HttpOnly`, `SameSite=Lax`; app bound to `127.0.0.1`.
- **Request → approve → expire:** pending request changes nothing; approval inserts an allow rule with `expires_at` at index 0; the proxy (Topic 2) makes it end.
- **Safe validation:** a requester may only ask for domains the proxy *currently denies for them* (`blocked_sites_for` uses the proxy's own matcher); no duplicate pending requests.
- **Talking to the proxy:** `admin_command()` → TLS + token → `RULE INSERT` / `RULE DEL` / `STATUS`.
- **Atomic file writes:** `write_json()` writes a temp file and `replace()` s it.
- **One layout, many pages:** `render_page()` + `base.html` so the sidebar badge always matches the data.

Walk through your code (say it in this order)

1. Login → `check_password_hash` → session holds `username` and `role`; a CSRF token is created.
2. `before_request` rejects any POST without the right token.
3. Requester submits **Unblock a website** → `create_site_request()` validates → writes a pending request.
4. Admin sees it on **Access requests** → `decide_request()` (`kind == "site"`) inserts the temporary allow rule → history entry.
5. **Grants** page shows the live countdown (`data-seconds` + `app.js`); `revoke_site_grant()` ends it early.

6. `purge_expired_real_rules()` tidies expired rules; the status light comes from `proxy_status()`.

Live demo

```
python3 app.py                          # http://127.0.0.1:5000
# admin / admin-demo  → Website access → add youtube.com, Block
# bob / bob-demo (private window) → My requests → Unblock a website → 5 minutes
# admin → Access requests → Approve  → Temporary grants shows the countdown
python3 -m unittest tests.test_site_temp_access tests.test_accesswindow_roles -v
python3 demo.py                         # scripted walk-through with PASS/FAIL checks
```

Open `packet_flow.html` and narrate the "Temporary unblock" scenario.

Hand-off lines

- *"How does the proxy decide?"* → "That's the policy engine — Topic 2. I only edit the rule list it reads."
- *"How is the TLS connection secured?"* → "That's the network core — Topic 1. My dashboard connects to it as an admin client over TLS with a token."

Your viva questions

Q29. How is the requester prevented from approving their own request?

Server-side `@role_required("admin")` on the approve/reject/revoke routes returns 403 — hiding the button isn't security.

Q30. What is CSRF and how do you prevent it?

A malicious site making your browser submit a form to the dashboard. Each session has a random token that every POST must include, compared with `hmac.compare_digest`.

Q31. How are passwords stored?

Salted PBKDF2-SHA256 hashes via Werkzeug, never plaintext; verified with `check_password_hash`.

Q32. Why did the app crash with `hashlib has no attribute 'scrypt'` and how was it fixed?

Werkzeug's default hash is scrypt, which this Python build (linked to LibreSSL) lacks. I set `method="pbkdf2:sha256"`.

Q33. Why bind to 127.0.0.1?

The dashboard is an admin interface using demo credentials; keeping it on loopback avoids exposing it to the network.

Q34. Why write JSON via temp file + `replace()` ?

It is atomic — the proxy reading the file at the same moment never sees a half-written document.

Q35. Walk me through requesting a site.

Request → admin approves → allow rule with `expires_at` inserted at the top → the proxy matches it → expiry makes it inert.

Q39. How do you stop a user requesting a site that isn't blocked?

`blocked_sites_for(user)` uses the proxy's own `match_rule` ; only domains it currently denies for that user are accepted.

Q40. Does approval affect other users?

No — the rule's `users` is `["bob"]` ; Carol is still blocked (there is a test).

Q41. How did you test it?

Unit tests for the rule engine, filters and domain matching; integration tests that start the whole stack over real sockets; role tests; and tests for temporary unblock (request → approve → expire, revoke, self-approval refused).

Q42. One known failing test?

`test_tls_and_identity` expects TLS 1.3; on this Mac the system Python negotiates TLS 1.2. It's an environment difference, not a logic bug.

Q45. What if two admins edit at once?

Writes are atomic but read-modify-write isn't locked across processes; a database or file lock would fix it.

Q47. How would you scale this?

Replace JSON with a database or cache, use async I/O or a process pool, central logging, and hashed credentials / SSO.

Q48. Why simulated services?

A safe, self-contained lab: reproducible demos and tests without touching real sites.

Q54. The Browse page fetches URLs for the user — how do you stop it being abused to reach internal addresses?

Only `http` / `https`; the host must resolve to a public IP (so localhost, the admin console and the LAN are refused); redirects are followed manually and every hop is re-checked against the policy and the IP rule; the preview runs in a sandboxed, scripts-off frame; login is required.

Shared — everyone answers these

Q1. What does your project do?

It filters network access through a proxy using ordered rules; a dashboard lets an admin manage rules and approve users' requests for temporary access, which expires automatically.

Q2. What is a proxy? Forward vs reverse?

A middleman between client and server. A *forward* proxy acts for clients (the client is configured to use it) — ours. A *reverse* proxy sits in front of servers (load balancers, CDNs).

Q3. Why use a proxy for filtering rather than filtering in the browser?

It is central, enforced outside the user's control, works for any app that honours the proxy, and gives one place to log and audit.

Q4. What's the difference between your two proxies?

`proxy.py` / `netproxy` is a TLS proxy for simulated lab services with identity, a rule engine and content filters. `forward_proxy.py` is a standard HTTP/HTTPS proxy for real domains with hostname filtering only.

Who did what? *"We split it into three: the network and security core, the policy and filtering brain, and the dashboard with the test lab. Each of us can demo and defend our own part, and we agreed the interfaces between them."*

Presenting together

Suggested order (about 3 minutes each): Topic 3 shows the dashboard and the request → approve flow → Topic 2 explains why the rule matches and then expires → Topic 1 shows what happens on the wire (TLS, identity, tunnel). Close with `packet_flow.html`.

If you're asked about someone else's part: give a one-sentence answer, then hand over by name — *"That's [Member N]'s part, they can explain it in detail."* Don't guess.

Checklist before the viva — each member: - [ ] I can read every file in my list and say what each function does. - [ ] I can run my demo commands from a cold start. - [ ] I can answer all my questions without notes. - [ ] I know the three hand-off sentences for the neighbouring parts. - [ ] I have read the other two parts' 30-second pitches.